Neo-AA: Cross-Ecosystem Account Abstraction with Heterogeneous Signature Verification

R3E network
For Neo Ecosystem

March 11, 2026

Abstract

We present Neo-AA, a Neo N3 account-abstraction architecture that unifies native Neo witness authorization with EVM EIP-712 typed signatures inside one shared master contract. The central design challenge is heterogeneity: Neo N3 authorizes transactions through secp256r1-backed witnesses, whereas Ethereum-style workflows authorize through secp256k1 signature recovery over typed data. Neo-AAbridges these models without deploying one wallet contract per user. Instead, it binds each logical account identifier to a deterministic abstract-account address that becomes the user-facing identity, while the logical `accountId` remains an internal coordination key for storage, indexing, and verification.

The system contributes four main elements. First, it introduces a mixed authorization protocol that counts Neo witnesses and recovered EVM signer addresses toward one threshold policy. Second, it provides deterministic address generation through a verify-proxy script that routes verification to a shared contract, eliminating per-account deployment costs. Third, it supports human-readable discovery through `.matrix` domain resolution and address-based account lookup. Fourth, it exposes a pluggable recovery architecture including Argent-style guardian sets, Safe-style timelocked recovery, Loopring-style guardian-hash recovery, and an oracle-gated dome inactivity flow. We describe the full architecture, state model, workflow, and security properties, and we align the specification with the current implementation and testnet validation pipeline.

1 Introduction

Account abstraction decouples user identity from cryptographic keys, enabling flexible authorization policies. While Ethereum's ERC-4337 and similar proposals provide single-chain solutions, cross-ecosystem authorization remains unsolved. Neo N3's witness-based transaction model and Ethereum's signature recovery present fundamentally different authorization primitives that cannot be unified through simple bridging.

1.1 Motivation

Traditional blockchain accounts tie identity directly to one signing key or to one per-chain smart-wallet deployment. This coupling complicates recovery, fragments user presence across ecosystems, and makes multi-wallet coordination operationally expensive. Neo N3 and EVM ecosystems exacerbate the problem because they expose different authorization primitives, different signing curves, and different transaction assembly workflows.

1.2 The Neo N3 Challenge

Neo N3 transactions use a witness array where each witness contains an invocation script and a verification script. Authorization occurs when the verification script (typically checking ECDSA signatures on secp256r1) validates the invocation script against the transaction hash. This model differs fundamentally from Ethereum's approach where signatures are recovered to addresses that are then checked against authorization lists.

Naive Approaches vs. Neo-AA

A naive cross-chain account abstraction typically requires:

- Deploying separate smart contracts on each chain.
- Synchronizing authorization state (e.g., via slow cross-chain messaging) across heterogeneous runtimes.

- Maintaining chain-specific signing flows with no cohesive, multi-chain threshold semantics.

These overheads are precisely what Neo-AA is designed to eliminate.

1.3 Challenges

Core Architectural Challenges

C1: Heterogeneous Signatures
Neo N3 uses witness-based authorization with secp256r1, while EVM wallets use EIP-712 typed data and secp256k1 recovery. A unified system must count both schemes without weakening either trust model.

C2: Deterministic Addressing
Per-account deployment is expensive and operationally noisy. A practical design needs stable account addresses without forcing one contract deployment per user.

C3: Address-First Usability
Users typically remember an account address or human-readable domain, not a logical identifier. The account model therefore needs a clean distinction between public account identity and internal storage keys.

C4: Recovery and Extensibility
Production accounts require recoverability, role separation, and policy mutation without hard-coding one recovery scheme into the core contract.

1.4 Our Solution: Mixed Authorization Protocol

Neo-AAintroduces a *mixed authorization protocol* in which a single Neo N3 master contract accepts both native Neo witnesses and EVM EIP-712 typed signatures and counts them toward one threshold. The key architectural observation is that both signer types can ultimately be normalized into addresses: Neo witnesses are checked with `Runtime.CheckWitness`, while EVM signatures are verified on-chain, mapped to recovered Ethereum-style addresses, and evaluated against the same stored role sets.

The Neo-AA Approach

Deterministic Addressing
Instead of deploying one contract per account, Neo-AAderives a deterministic verify-proxy script that forwards verification to the master contract. Internally, each account is keyed by a logical `accountId`; externally, users interact with the deterministic abstract-account address produced from that proxy script.

Address-First Routing
The common execution path exposes both logical-id entrypoints and address-based convenience wrappers. User-facing interfaces rely exclusively on the deterministic AA address—or a bound `.matrix` domain—rather than an opaque logical identifier.

EIP-712 Integration
EVM signers authorize an operation by signing typed data bound to the AA address, target contract, method hash, argument hash, nonce, and deadline. The master contract reconstructs the digest on-chain, recovers signer addresses, and feeds them into the same permission engine used for Neo execution.

1.5 Contributions

This paper makes the following contributions:

Key Contributions

Mixed Authorization Protocol
A unified threshold mechanism that counts Neo witnesses and EVM EIP-712 signatures toward one authorization threshold, enabling cross-ecosystem account abstraction without bridge contracts or synchronized replica state.

Deterministic Address Generation
A zero-deployment account model where verify scripts are deterministically generated and route to a shared master contract, eliminating per-account deployment costs while maintaining address stability.

Unified Threshold Algorithm
The `CheckMixedSignatures` algorithm seamlessly integrates Neo's `Runtime.CheckWitness` with EVM signature recovery, supporting pure Neo, pure EVM, and mixed authorization scenarios through a single verification path.

Formal Security Analysis
Rigorous security definitions and proofs demonstrating threshold security, replay protection, and authorization correctness under the standard cryptographic assumptions for both signature schemes.

EVM Integration and Address-First Architecture

How EVM Participation Works
EVM wallets join Neo-AA through EIP-712 typed-data signatures. When an EVM signer authorizes an operation, the signed message commits to the AA account address, target contract, method hash, argument hash, nonce, and deadline. The signature is carried into a Neo transaction, verified on-chain with secp256k1/Keccak256, and converted into a recovered Ethereum address that is evaluated against the stored admin or manager sets.

Address-First Account Management
The contract still stores a logical `accountId`, but the dominant external interface is address-based. The common creation path is `createAccountWithAddress`, which binds the logical record directly to its deterministic proxy address. The contract exposes bidirectional lookup methods like `getAccountIdByAddress`.

Role Discovery and Indexing
Instead of forcing users to remember internal identifiers, the master contract maintains address-centric indexes. The contract natively exposes role-specific queries like `getAccountAddressesByAdmin`.

1.6 .matrix Domain Integration

Neo-AAintegrates with the Matrix naming system as a discovery layer, not as an authorization root. When a user enters `alice.matrix`, the frontend resolves the domain owner from the Matrix contract, queries the AA contract for bound addresses associated with that owner, and presents the discovered AA accounts for selection. A typical resolution flow is:

Matrix Domain Lookup Flow

1. Resolve the `.matrix` domain to its owner address through the Matrix contract.
2. Query `getAccountAddressesByAdmin` and `getAccountAddressesByManager` on the AA contract using the resolved owner address.
3. Merge, de-duplicate, and present the resulting bound AA addresses.
4. Load the selected AA account and continue all subsequent execution through the account address.

This separation is intentional: domains improve usability and discovery, while the authoritative authorization state remains on-chain in the AA contract's role and policy storage.

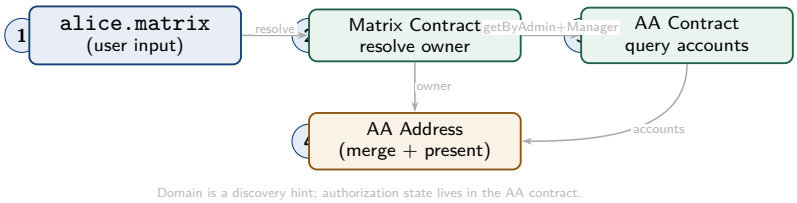

Figure 1: `.matrix` domain resolution flow. The domain resolves to an owner address, which is used to query the AA contract for bound accounts.

1.7 Recovery Verifier Architecture

Neo-AA supports pluggable recovery verifiers through a standardized interface. Each account can optionally set a recovery verifier contract that implements:

```
interface IRecoveryVerifier {
  bool verifyRecovery(
    byte[] accountId,
    byte[] operation,
    byte[] proof
  );
}
```

Argent-Style Guardian Recovery

The Argent recovery model implements a social recovery mechanism where trusted guardians can collectively authorize account recovery operations.

On-Chain State Maintained

- **Guardian Set**: Serialized array of guardian addresses, stored as `byte[]` to support both Neo and EVM addresses.
- **Guardian Threshold**: Minimum number of guardian signatures required (m out of n guardians).
- **Pending Operations**: Map of recovery operation hashes to pending recovery requests.
- **Timelock Period**: Configurable delay (default: 24 hours) between initiation and execution.

Recovery Operation Lifecycle

Phase 1: Initiation

A guardian initiates recovery via the entrypoint by supplying the `accountId`, replacement administrators, and the proposed threshold. The mechanism computes a hash $h = H(\text{accountId}\|\text{newAdmins}\|\text{newThreshold}\|\text{nonce})$, stores the pending operation, and emits a `RecoveryInitiated` event.

Phase 2: Guardian Approval

Additional guardians approve the pending operation by submitting the operation hash alongside their approval signatures. The verifier validates the signature against the hash and increments the approval count.

Phase 3: Execution

Once the threshold is met and the timelock expires ($\text{Runtime.Time} - \text{initiationTime} \geq T_{\text{lock}}$), anyone can call `executeRecovery(operationHash)`. The verifier confirms the conditions and triggers the master contract update.

Cancellation

The current account owner can reset and cancel any pending recovery operation at any time by successfully performing a standard authorized transaction.

Safe-Style Threshold Timelock

Extends the core Argent model with enhanced timelock flexibility and modular execution.

Key Design Elements

- **Configurable Timelock**: Each account can set custom timelock periods (1 hour to 30 days).
- **Execution Window**: After the timelock T_{lock} expires, the operation remains valid strictly for a time window T_{window} (default: 7 days).
- **Multi-Step Recovery**: Supports partial admin replacement for a gradual transition rather than complete takeover.

The Safe verifier ensures execution fails if Runtime.Time $>$ initiationTime $+ T_{\text{lock}} + T_{\text{window}}$.

Loopring-Style Guardian Hash

Prioritizes privacy by hiding guardian identities until the recovery event, storing only a commitment hash on-chain: $H_g = H(\text{guardian}_1 \| \ldots \| \text{guardian}_n \| \text{threshold})$. The verifier also maintains a `Recovery Nonce` to prevent replay attacks across multiple recovery attempts.

Recovery with Hash Revelation

Phase 1: Initiation

The initiator calls `initiateRecovery(accountId, guardianSet, threshold, signatures)`. The verifier computes $H_{\text{revealed}} = H(\text{guardianSet} \| \text{threshold})$ and confirms $H_{\text{revealed}} = H_g$. It then verifies signatures and stores the pending operation.

Phase 2: Execution

After the timelock, `executeRecovery` verifies the revealed guardian set matches the stored hash and executes the recovery operation.

This design guarantees that guardian addresses cannot be enumerated or analyzed before an actual compromise event forces a recovery.

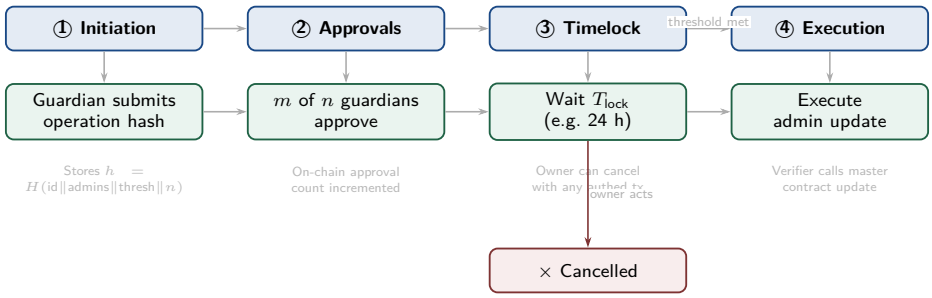

Figure 2: Guardian recovery lifecycle (Argent-style). Four phases protect recovery operations: initiation commits the hash, guardian approvals accumulate, a timelock gives the owner a cancellation window, then execution updates roles.

1.8 Dome Inactivity Recovery Protocol

The dome mechanism provides last-resort recovery for accounts with prolonged inactivity.

Dome Inactivity Flow

Inactivity Detection

The system tracks the last activity timestamp for each account: lastActivity[accountId] $=$ Runtime.Time. An

account is flagged as inactive if Runtime.Time − lastActivity $> T_{\text{inactivity}}$, where $T_{\text{inactivity}}$ is highly configurable (default: 180 days).

Oracle-Gated Initiation

A designated recovery address can initiate dome recovery, but execution requires external oracle approval:

```
function initiateDomeRecovery(accountId) {
  require(msg.sender == domeRecoveryAddr[accountId]);
  require(isInactive(accountId));

  requestId = hash(accountId, Runtime.Time);
  pendingRecovery[requestId] = {
    accountId: accountId, initiator: msg.sender,
    timestamp: Runtime.Time, status: "pending_oracle"
  };

  emit DomeRecoveryInitiated(requestId, accountId);
}
```

Oracle Validation & Recovery

An authorized oracle verifies genuine inactivity off-chain and signs an approval. After oracle approval, a final timelock period (default: 7 days) must elapse. During this window, the original owner can unilaterally cancel the recovery by performing any authorized transaction.

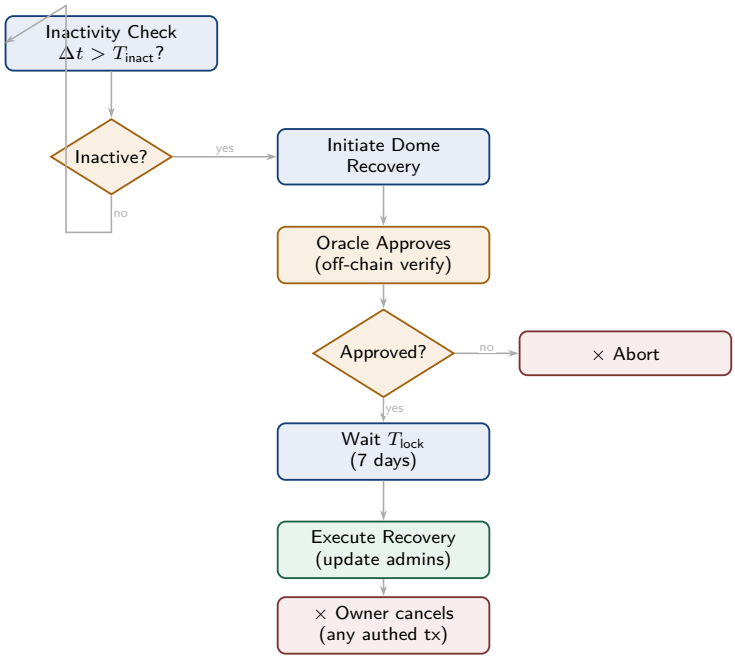

Figure 3: Dome inactivity recovery flow. An inactive account can be recovered via a designated address, subject to oracle approval and a 7-day cancellation window for the original owner.

2 Account Model and Lifecycle

2.1 Account Creation and Structure

An abstract account in Neo-AAis a shared-contract wallet instance with two identifiers: a logical `accountId` and a deterministic AA address. The `accountId` is the internal storage key and verification parameter; the deterministic AA

address is the public, user-facing identity. This separation is central to the implementation and to the user experience: users fund, discover, and invoke the AA through its address, while the contract keeps the logical identifier for compact indexing, address derivation, and compatibility with internal verification routines.

Identity Model

Logical identity: `accountId` is created once, validated, and used as the canonical key for durable storage inside the master contract.

Operational identity: The deterministic AA address is the signer identity that wallets, relayers, and downstream contracts query and interact with.

Binding layer: The master contract maintains authoritative bidirectional `accountId` ↔ `accountAddress` mappings.

User-facing rule: After creation, all operational pathways strongly prefer the AA address; `accountId` is explicitly retained for initialization, storage indexing, and base verifier compatibility.

Account Creation Flow.

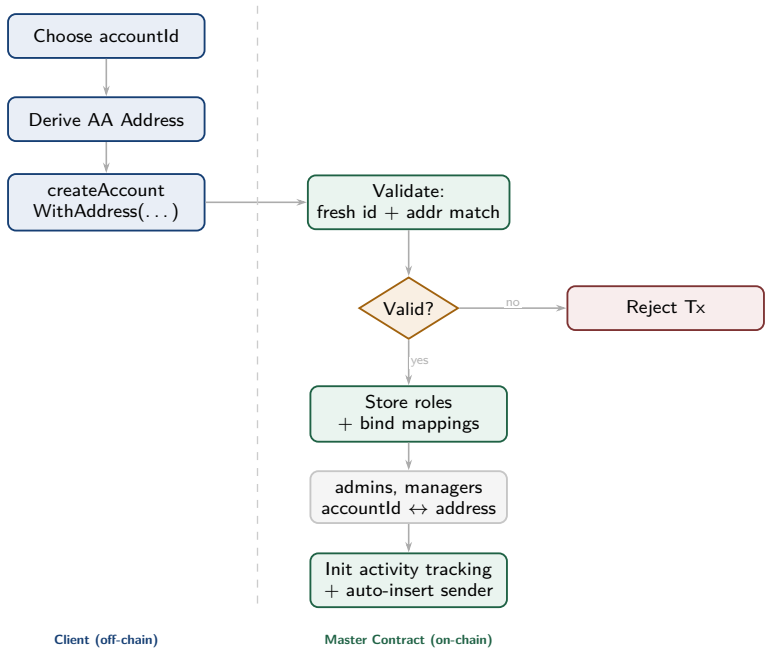

Figure 4: Account creation and binding lifecycle. The client derives the deterministic address from `accountId`, then the master contract validates uniqueness, stores role sets, and binds bidirectional address mappings.

1. The client chooses an `accountId` and deterministically derives the corresponding AA verify-proxy address.
2. The common creation path invokes the address-binding creation wrapper `createAccountWithAddress(...)`.
3. The master contract validates that the logical identifier is fresh, the supplied address matches the deterministic proxy derivation, and the role thresholds are well formed.
4. The contract stores the admin and manager sets under the canonical storage key for `accountId`, initializes activity tracking, and binds the bidirectional address mappings.
5. The transaction sender is automatically inserted into the admin set when not already present, ensuring that bootstrap creation cannot strand the account.

The contract also supports `createAccountBatch` for batched logical-account initialization under shared role parameters. In that mode, the contract still validates role thresholds and still defaults the sender into the admin set when no explicit administrators are supplied. Batch creation is therefore an efficiency tool, not a separate trust model.

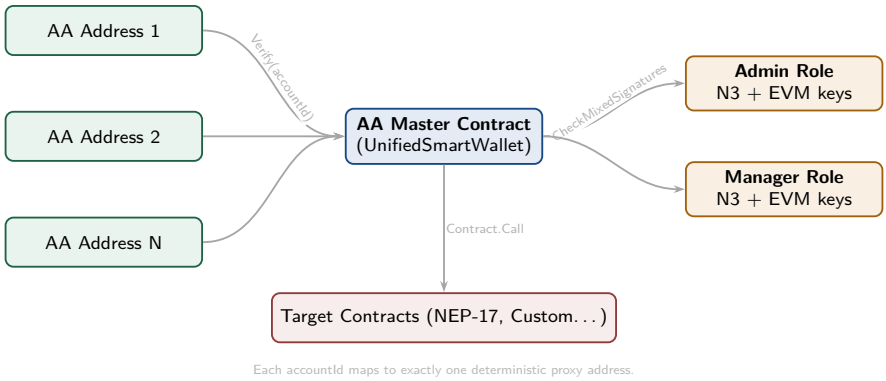

Figure 5: Neo-AA system architecture. A shared master contract governs many deterministic AA addresses while storing role sets and authorization policies in one on-chain control plane.

2.2 Deterministic Address Generation

Each abstract account has a deterministic Neo address derived from its `accountId` and the master contract hash. That deterministic address is the account's public identity, signer handle, and discovery key.

Verify-Proxy Construction.

The exact bytecode is produced by the contract's `BuildVerifyProxyScript(accountId)` helper. At a logical level, the proxy script performs the following steps:

```
push accountId
push "verify"
push masterContractHash
syscall System.Contract.Call
```

Conceptually, the proxy script:

1. Carries the logical `accountId` into the verification call.
2. Routes verification to the shared AA master contract rather than to a per-account contract instance.
3. Causes the script hash of the proxy to become the externally visible AA address.
4. Allows downstream contracts to rely on `Runtime.CheckWitness(accountAddress)` once an authorized AA execution path has opened the proper verification context.

Deterministic Proxy Addressing

The Neo address is computed as:

$$\text{address} = \text{Base58Check}(\text{version} \| \text{Hash160}(\text{verifyScript}))$$

where `version = 0x35` for Neo N3. The script hash is derived from the proxy bytecode and reversed into Neo's `UInt160` representation before Base58Check encoding.

Key Properties

Deterministic Construction

The same `accountId` always yields the exact same verify-proxy address.

Zero Deployment Cost

No per-account contract deployment transaction is required.

Address-First Usability

The account can be funded, discovered, and referenced by address immediately after derivation.

Binding Integrity

The master contract refuses to bind an `accountId` to any address other than its mathematically derived proxy address.

2.3 Transaction Structure and Verification Entry

Transaction Integrity Rule

All transactions using an abstract account MUST include the deterministic verify address as a signer. This strictly triggers the verification entry point in the master contract.

Standard Neo-Only Transaction Structure

1. **Script**: Invokes the address-based wrapper `executeByAddress(accountAddress, targetContract, method, args)`.
2. **Signers**: Array containing the deterministic verify address (script hash).
3. **Witnesses**: Array of witness objects containing an empty Invocation Script (or signature data) and the deterministic Verification Script.

Verification Entry Flow

1. **VM Check** Node executes the verification proxy script, calling `masterContract.verify(accountId)`.
2. **Guard** The master contract ensures the calling context is a valid self-invocation targeting an approved wrapper.
3. **Application Phase** If verified, the transaction script executes the wrapper method.
4. **Authorization** The wrapper performs strict authorization checks (admin/manager thresholds) against the on-chain policy.
5. **Dispatch** If authorized, the wrapper safely invokes the target contract.

This two-phase design guarantees that the deterministic verify address cannot be misused for unauthorized external calls, and all operations pass through the master contract's centralized policy enforcement layer.

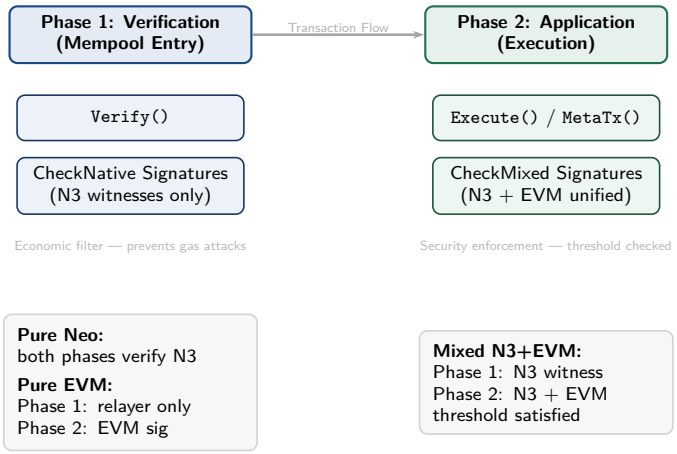

Figure 6: Two-phase verification in Neo-AA. The verification trigger authenticates native Neo paths, while the application trigger combines Neo and EVM signers for mixed authorization.

2.4 Mixed Authorization Transaction Anatomy

Mixed authorization transactions combine Neo witnesses with EVM EIP-712 signatures. The transaction structure extends the standard Neo transaction with embedded EVM signature data.

Mixed Transaction Structure

1. **Script**: Invokes the address-based meta-transaction wrapper `executeMetaTxByAddress(...)`.
2. **Signers**: Deterministic verify address (same as Neo-only transactions).
3. **Attributes**: Custom transaction attribute containing EVM signature (r, s, v), Nonce, and Deadline.
4. **Witnesses**: Neo witnesses (may be empty if only EVM authorization is used).

EVM Signature Verification Flow

The master contract's `executeMetaTxByAddress` method evaluates EVM payloads via the following sequence:

1. **Reconstruct** Builds the EIP-712 typed data hash from the provided parameters.
2. **Recover** Extracts EVM public key: $\text{pubkey} = \text{ecrecover}(\text{hash}, r, s, v)$.
3. **Derive** Computes the Ethereum address: $\text{addr}_{\text{EVM}} = \text{Keccak256}(\text{pubkey})[12:32]$.
4. **Validate Anti-Replay** Checks nonce matches n_{stored} and time $\leq$ deadline.
5. **Map to Roles** Counts recognized EVM signatures alongside any Neo witnesses toward the required threshold.

Neo vs. Mixed Transaction Verification

While pure Neo transactions authenticate during the memory pool phase, mixed transactions introduce EVM signatures via transaction attributes that must be verified during execution.

- **Payload Location**: EVM signatures are explicitly embedded as parameters within the execution script, not inside the native Neo witness array.
- **Timing**: EVM signatures are verified securely on-chain during the application phase, preventing manipulation of the abstract account state.
- **Replay Protection**: EVM payloads utilize deliberate, account-scoped nonces embedded in the EIP-712 hash, whereas Neo witnesses are intrinsically bound point-in-time to the transaction hash.
- **Unified Policy**: Both paths route into one ultimate counting process against the master policy threshold.

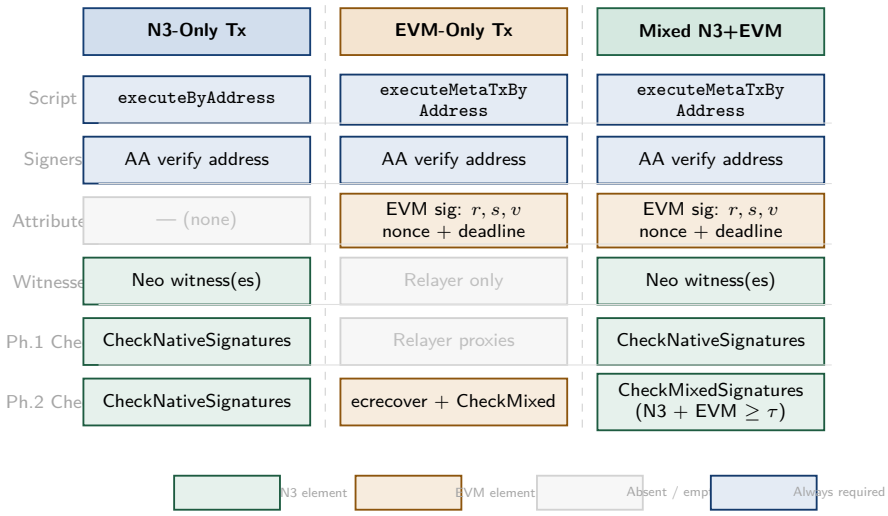

Figure 7: Transaction layout comparison for the three authorization modes. Script and signer fields are always required. EVM-only and mixed modes embed the signature in a transaction attribute; N3-only uses only Neo witnesses. The Phase 2 check escalates from native-witness-only to the full unified `CheckMixedSignatures` path.

3 Mixed Authorization Protocol

3.1 Protocol Overview

Mixed Authorization Protocol Phases

Phase 1: EVM Signature Generation
(Off-chain). The EVM signer constructs EIP-712 typed data and signs with their Ethereum wallet. The signature is a 65-byte value (r, s, v) where $r, s \in \mathbb{F}_p$ are ECDSA signature components and $v \in \{27, 28\}$ is the recovery identifier.

Phase 2: Transaction Construction
A Neo transaction is built with:

- **Script**: `CALLT` to master contract wrapper method
- **Signers**: Deterministic verify address for the account
- **Attributes**: EVM signature embedded as custom transaction attribute
- **Witnesses**: Placeholder for Neo signers

Phase 3: Neo Witness Collection
Neo signers add witnesses to the transaction. Each witness contains an invocation script (typically empty or containing the signature) and a verification script (the deterministic proxy script).

Phase 4: On-Chain Verification
The master contract validates the deterministic proxy witness, counts Neo witnesses, recovers the Ethereum addresses from the EVM signatures, and checks if the combined signature payload hits the unified operational threshold.

3.2 Core Verification Methods

The Abstract Account contract implements three fundamental verification methods that form the foundation of the mixed authorization protocol. These methods handle different verification scenarios and execution phases.

3.2.1 Verify Method: Two-Phase Verification Model

The `Verify(accountId)` method is the unified verification entrypoint called by the deterministic proxy script. It implements a two-phase verification model that distinguishes between transaction validation and application execution.

Method Signature:

```
public static bool Verify(ByteString accountId)
Location: AbstractAccount.AccountLifecycle.cs:66-113
```

Two-Phase Verification Architecture

The Neo VM verification system operates in two distinct phases, mapping naturally to our mixed-signature routing:

Phase 1: Verification Trigger (Transaction Validation Phase)

- **Timing**: Occurs when a transaction enters the memory pool, before application execution.
- **Limitation**: EVM signatures cannot be verified here because EIP-712 signature data resides in transaction parameters, which are inaccessible during the pure Verification phase.
- **Behavior**: The `Verify` method checks if the transaction shape matches the standard AA wrapper call, then exclusively counts Neo `CheckWitness` signatures.

Phase 2: Application Trigger (Execution Phase)

- **Timing**: Occurs during the execution phase when the target contract explicitly calls `Runtime.CheckWitness(accountAddress)`.
- **Behavior**: `VerifyContext` ensures only the currently pre-authorized target contract can pass, unlocking EVM typed verification logic.

3.2.2 CheckNativeSignatures: Pure Neo Signature Verification

The `CheckNativeSignatures` method handles pure Neo signature verification using `Runtime.CheckWitness()`. It is used in the Verification phase and in pure Neo execution paths.

Method Signature:

```
private static bool CheckNativeSignatures(
    List<UInt160> roles,
    int threshold
)
Location: AbstractAccount.ExecutionAndPermissions.cs:252-261
```

Pure Neo native Verification

The `CheckNativeSignatures` method represents the optimized code path used strictly for pure Neo workflows where no EVM payload parsing is required.

Usage Scenarios

Called during the initial Verification phase (transaction validation) filtering to efficiently reject unauthenticated payloads early, and inside the Application execution phase for pure native operations without meta-data overhead.

Mechanism and Limitations

Leverages `Runtime.CheckWitness` directly on the node layer. Since it only targets the signature evidence loaded in the `Witnesses` array, it mathematically cannot evaluate EVM typed-data signatures encoded into execution arguments.

3.3 CheckMixedSignatures: Unified Threshold Algorithm

The core innovation of the mixed authorization protocol is the `CheckMixedSignatures` algorithm, which unifies Neo witness verification and EVM signature verification into a single threshold check. This algorithm is implemented in `AbstractAccount.ExecutionAndPermissions.cs:281-315`.

Algorithm Definition:

```
CheckMixedSignatures(roles, threshold, verifiedSigners):
  Input:  roles          - List of authorized addresses (UInt160[])
          threshold      - Minimum required signatures (int)
          verifiedSigners - EVM addresses recovered from signatures (UInt160[])
  Output: Boolean indicating whether threshold is satisfied

  count <- 0
  for each address in roles:
    matched <- false

    // Step 1: Check if address is an EVM signer
    if verifiedSigners != null:
      for each signer in verifiedSigners:
        if address == signer:
          count <- count + 1
          matched <- true
          break
```

```
    // Step 2: Check if address is a Neo signer
    if not matched:
      if Runtime.CheckWitness(address):
        count <- count + 1

    // Early exit if threshold reached
    if count >= threshold:
      return true

  return false
```

Unified Counting Mechanics

Cryptographic Isolation

EVM signatures are authenticated independently using secp256k1 and Keccak256 logic inside the contract application context. Neo witnesses leverage the node's native secp256r1 core mechanism. The two validation paths cannot intercept, override, or spoof one another.

Equitable Contribution

Regardless of their origin, each cryptographically validated key mapped to a known account role contributes exactly one unit of authorization weight against the unified threshold.

Order Independence

The counter algorithm loops over the canonical, on-chain stored array of authorized roles, evaluating valid presence within either the recovered EVM signing set or the Neo environment. This ensures policy behavior is completely agnostic to payload insertion order.

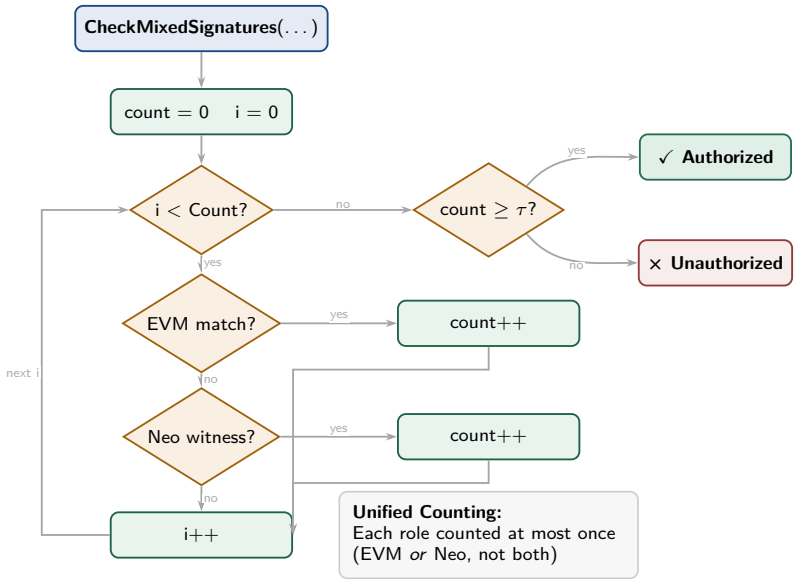

Figure 8: Unified threshold evaluation in CheckMixedSignatures. Recovered EVM signers and Neo witnesses are counted against one policy threshold.

3.4 EIP-712 Signature Verification Details

The EIP-712 verification process ensures that EVM signatures are cryptographically bound to the specific operation and cannot be replayed or misused.

Domain Separator Construction:

$$\begin{aligned}\text{domainSep} = \text{keccak256}(&\text{EIP712_DOMAIN_TYPEHASH}\\ &\|\text{keccak256}(\text{"Neo N3 Abstract Account"})\\ &\|\text{keccak256}(\text{"1"})\\ &\|\text{uint256}(\text{chainId})\\ &\|\text{address}(\text{verifyingContract}))\end{aligned}$$

where chainId = 894710606 for Neo N3 testnet and verifyingContract is the master contract hash.

3.5 State Management and Nonce Mechanism

The mixed authorization protocol maintains an *account-scoped* meta-transaction nonce. Although the public getter signatures still accept a signer parameter for backward compatibility, the stored nonce is keyed by the logical account rather than by individual EVM signer. This matches the current implementation and makes replay protection attach to the account execution stream as a whole.

Nonce and Activity State.

For each abstract account, the master contract stores:

- **Meta-transaction nonce:** a single monotonically increasing value under the account's nonce storage key.
- **Last activity timestamp:** updated whenever the account successfully satisfies an authorization path.
- **Temporary execution context:** scoped verify-context and recovered-signer context used only during an active execution path.

Nonce Validation Flow.

When processing an EVM meta-transaction, the master contract:

1. Resolves the logical account from the supplied AA address when the address-based wrapper is used.
2. Recomputes the expected `argsHash` and EIP-712 digest for that account and target call.
3. Reads the account-scoped stored nonce and requires the provided nonce to match exactly.
4. Verifies the provided secp256k1 signatures, derives recovered signer addresses, and only then increments the stored account nonce before dispatch.

This account-scoped design intentionally serializes signed meta-transaction execution for a given AA account. Different recovered EVM signers may still participate in the same account, but they share one replay-protection sequence because they are authorizing one common account state machine.

3.6 EIP-712 Typed Data Structure

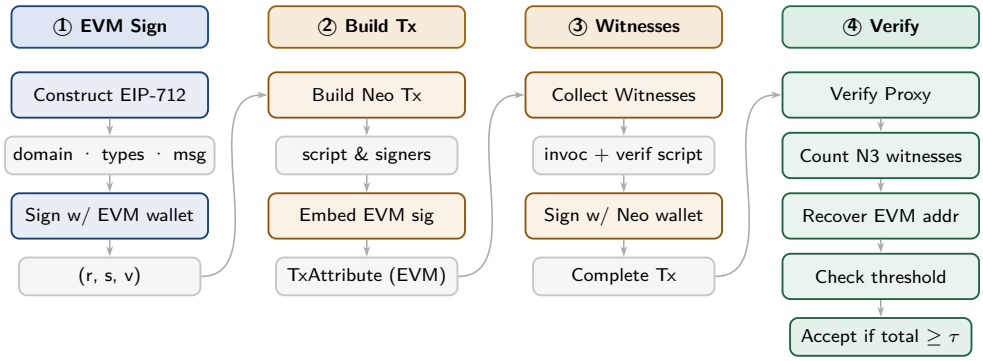

Figure 9: Mixed authorization workflow combining EVM typed-data signing with Neo execution under one threshold.

The exact typed-data structure used by the frontend and SDK is:

```
EIP712Domain {
  name: "Neo␣N3␣Abstract␣Account"
```

```
  version: "1"
  chainId: <Runtime.GetNetwork()>
  verifyingContract: <masterContractHash>
}

MetaTransaction {
  accountAddress: address
  targetContract: address
  methodHash: bytes32
  argsHash: bytes32
  nonce: uint256
  deadline: uint256
}
```

Two details are important. First, the signed identity is the AA *address*, not the raw `accountId`. Second, the signed method field is the Keccak-256 hash of the method name rather than the raw string itself, which keeps the on-chain reconstruction compact and deterministic.

The `argsHash` is computed by serializing the method arguments with Neo's `StdLib.Serialize` and hashing the serialized bytes with Keccak-256, exactly matching the contract's `computeArgsHash` helper. After successful verification, the account-scoped nonce is incremented: $n \leftarrow n + 1$.

The `deadline` is normalized to Neo's millisecond-based `Runtime.Time`. Frontends often sign unix-second deadlines; the contract accepts those inputs by normalizing them before comparison and rejects the meta-transaction whenever:

$$\text{Runtime.Time} > \text{deadline}_{\text{normalized}}$$

3.7 Verification Algorithm

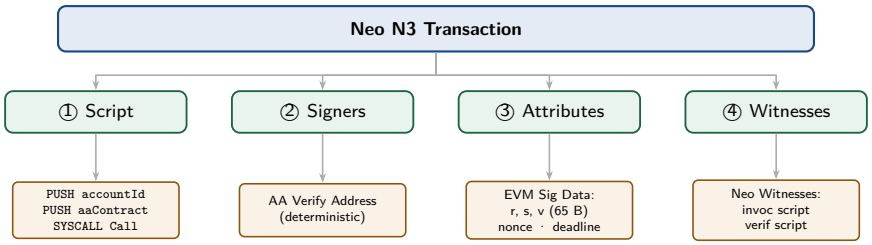

Figure 10: Neo N3 transaction layout for AA execution, highlighting the verify-proxy signer, optional EVM signature payloads, and Neo witnesses.

High-Level Threshold Algorithm

1. Initialize count = 0.
2. For each Neo witness: if the derived address is in the admin/manager set, increment count.
3. If an EVM signature is present: recover the public key, derive the Ethereum address, and if it exists in the admin/manager set, increment count.
4. Accept the transaction if and only if count $\geq$ threshold.

4 Security Analysis

4.1 Threat Model

The Adversary $\mathcal{A}$ is assumed to control the network messaging layer, can observe all mempool transactions, and can submit arbitrary payloads, but computationally cannot:

- Break the discrete logarithm assumption.

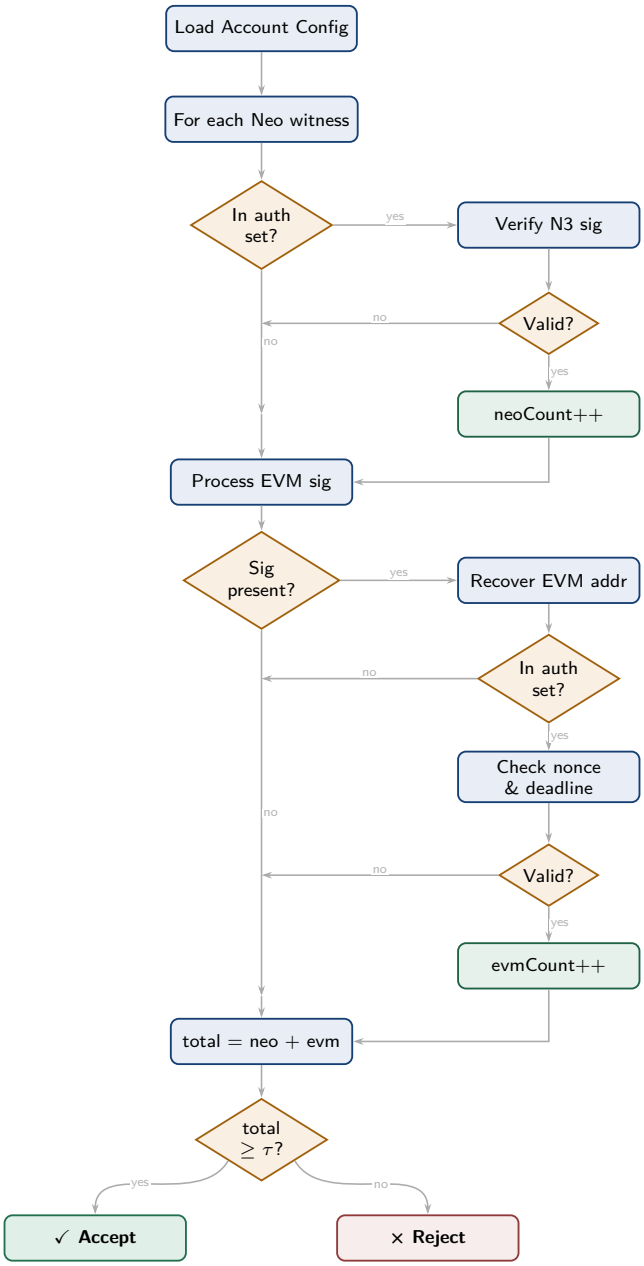

Figure 11: Threshold-verification workflow used by Neo-AA to load account policy, count signer evidence, and accept or reject execution.

- Forge ECDSA signatures (on either secp256k1 or secp256r1 curves) without private keys.
- Find hash collisions in SHA-256 or Keccak-256.

4.2 Formal Security Definitions

Definition 4.1 (Signature Unforgeability Game)**.** *Adversary $\mathcal{A}$ wins the game if:*

1. *$\mathcal{A}$ receives target public keys $\{pk_1, \ldots, pk_n\}$.*
2. *$\mathcal{A}$ makes adaptive signing queries $q_1, \ldots, q_m$.*
3. *$\mathcal{A}$ outputs a message and signature set $(m^*, \{\sigma_i^*\})$ such that no query was made for m^*.*
4. *Verification accepts $(m^*, \{\sigma_i^*\})$ while having less valid signatures than the threshold τ (e.g., $|\{\sigma_i^*\}| < \tau$).*

The advantage is defined as: $Adv_{\mathcal{A}} = \Pr[\mathcal{A}\ wins]$.

We define the security goals of Neo-AA through a series of structural assumptions and derived theorems. Table 1 summarizes the primary security constraints and how they are satisfied by the design.

Property	Enforcement Mechanism
Determinism	For a fixed input state (accountId, signatures, operations), execution outcome is strictly predictable across all nodes. Enforced by Neo N3 VM deterministic execution and shared contract routing.
Unforgeability	An adversary cannot fake an authorization payload. Enforced by discrete logarithm assumption on both secp256r1 (Neo witnesses) and secp256k1 (EVM ecrecover).
Replay Resistance	A captured signature cannot be re-exercised. Nonce mechanisms prevent cross-time replay. Neo witnesses bind to the unique transaction hash. EVM signatures bind to an account-scoped sequence nonce.
Soundness	An authorization containing less than the predefined τ valid signatures is strictly rejected. Checked automatically within the `CheckMixedSignatures` procedure.
Forward Security	A compromised signer key cannot rewrite past historical events. The strictly monotonic nonce advancement explicitly invalidates any attempt to sign backward in time.

Table 1: Summary of primary security properties underpinning the Neo-AA account abstraction protocol.

Theorem 4.1 (Threshold Correctness)**.** *Under standard ECDSA collision-resistance assumptions, the master contract's authorization engine accepts an operation if and only if the valid and unique signer count $n \geq \tau$.*

Proof Sketch. The `CheckMixedSignatures` procedure normalizes signers into a single domain by evaluating the set $\mathcal{R}_A \cup \mathcal{R}_M$. For Neo witnesses, `CheckWitness` guarantees that the witness contains a valid signature. For EVM signatures, the contract derives $H(m)$ and ecrecover confirms exactly one Ethereum address per signature tuple (r, s, v). Because the mapping step prevents duplicates, the contract accumulates exactly the count of verified members toward the threshold limit. □

Theorem 4.2 (Replay Bound)**.** *An intercepted EVM signature $\sigma = (r, s, v)$ constructed for state St_i cannot be re-applied to construct a valid evaluation at state St_j where $j > i$.*

Proof Sketch. The EIP-712 MetaTransaction payload binds σ to an account nonce n_i. Upon successful verification of the execution block in St_i, the contract strictly enforces $n_{i+1} = n_i + 1$. Any subsequent evaluation $St_j (j > i)$ will demand $n_j \geq n_{i+1}$. Since $n_i \neq n_j$, the signature verification on σ will fail on the fresh typed data hash. □

5 Related Work

5.1 Account Abstraction Systems

ERC-4337. Ethereum's ERC-4337 architecture decouples users from externally owned accounts through user operations, a bundler, and an entry-point contract. It is an important reference point for programmable wallet behavior,

but it assumes an Ethereum-native execution model and usually relies on per-account wallet contracts or account-specific validation logic. By contrast, Neo-AAconcentrates authorization in a shared Neo contract and is designed explicitly for heterogeneous Neo/EVM signer sets.

Argent and Safe. Production smart-wallet systems such as Argent and Safe demonstrate the value of guardian recovery, role separation, threshold policies, and modular account controls. Their relevance to Neo-AAis architectural rather than literal: they inform the recovery and governance surface, but they remain single-ecosystem designs and do not solve the problem of counting Neo witnesses and EVM typed signatures inside one threshold rule.

5.2 Threshold and Cross-Ecosystem Authorization

Threshold-signature literature. Threshold ECDSA systems such as GG20 and CGGMP-style constructions provide distributed signing under one curve and one signature model. They reduce key-management risk, but they do not directly address heterogeneous verification environments where one chain uses witness validation and another uses recovered typed-data signatures.

Bridge and interoperability systems. Most cross-chain systems focus on message passing or asset transfer. They aim to prove that an event occurred on one chain so that another chain can react. Neo-AAinstead targets authorization interoperability: one account policy should accept signers originating from different wallet ecosystems without forcing those ecosystems to share a signature format or a bridging trust assumption.

5.3 Positioning of Neo-AA

Relative to the above systems, Neo-AAoccupies a distinct design point: zero per-account deployment on Neo N3, deterministic address generation, mixed-signature threshold counting, and a recovery layer that is extensible without changing the primary execution interface. Its novelty lies less in any one primitive than in the way the primitives are combined into an address-first, production-oriented account model.

6 Discussion

6.1 Limitations

Current System Limitations

Curve Coverage

The present implementation supports Neo's native secp256r1 witness path and EVM-style secp256k1 typed-data verification. Supporting additional ecosystems (e.g., Ed25519) would require new signature-verification routines and new serialization or domain-separation rules.

Serialized Meta-Transaction Sequencing

The current account-scoped nonce is simple and safe, but it also means that meta-transactions for one account advance through one shared replay-protection sequence. This is a conscious safety-oriented choice rather than an oversight.

Execution Cost

EVM public-key recovery and signature verification add computational overhead relative to pure Neo witness validation. The design remains practical for the intended governance and account-operation workflows, but it is not entirely free.

6.2 Design Trade-offs

Architectural Trade-offs

Shared Contract vs. Per-Account Deployment

A shared contract minimizes deployment cost and simplifies upgrades to common logic, but pushes more complexity into deterministic routing, scoped execution contexts, and careful internal state separation.

Address-First Usability vs. Logical-ID Storage

Exposing the AA address as the primary public identity improves user experience and frontend ergonomics, while retaining `accountId` internally keeps storage, indexing, and verifier integration straightforward. The specification treats identifiers as complementary.

Unified Verification vs. Separate Execution Planes

Counting Neo witnesses and EVM recovered signers in one threshold path yields a simple and expressive policy model, but requires a two-phase verification loop and careful handling of temporary script contexts.

6.3 Operational Architecture

Modular Contract Structure

The deployed master contract follows a deliberately modular split to maintain maintainability despite its shared nature:

- **`AccountLifecycle`**: Account creation, validation, and deterministic address binding.
- **`StorageAndContext`**: Canonical storage keys, address mappings, activity tracking, verify context, and execution locks.
- **`ExecutionAndPermissions`**: Native execution, mixed authorization, policy enforcement, and contract dispatch.
- **`MetaTx`**: EIP-712 digest construction, argument hashing, nonce management, and recovered-signer handling.
- **`Admin Modules`**: Role mutation, dome configuration, oracle callbacks, and optional external verifiers.

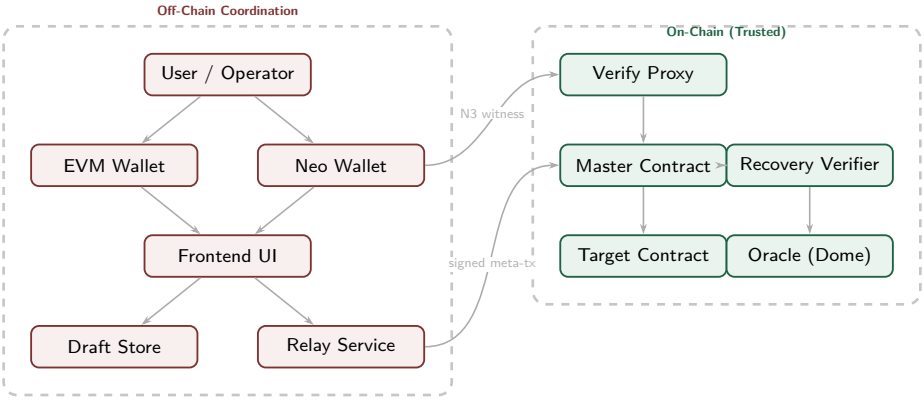

Figure 12: Trust-boundary view of Neo-AA. Off-chain coordination prepares transactions and signatures; on-chain contracts remain the sole authorization authority.

6.4 Future Directions

Promising extensions include broader ecosystem support (for example, additional signature curves and domain formats), privacy-preserving authorization with zero-knowledge techniques, richer batch execution patterns, and machine-checked verification of the full execution-context invariants. Another natural direction is stronger domain-centric discovery so that `.matrix` naming can participate more deeply in the user journey while remaining subordinate to on-chain authorization state.

6.5 Deployment Details

The production-oriented implementation is validated against Neo N3 testnet workflows and is accompanied by reproducible contract artifacts, integration scripts, and frontend support for collaborative transaction preparation. For

anonymous review we omit live identifiers, but the deployment model assumes one master contract, deterministic AA address derivation, address-based execution entrypoints, and recovery-verifier artifacts compiled alongside the core contract.

7 Conclusion

We presented Neo-AA, a Neo N3 account-abstraction architecture that unifies native witness-based authorization with EVM EIP-712 typed signatures inside one shared master contract. The core contribution is not merely signature verification, but a complete account model that combines deterministic AA addressing, address-first user interaction, mixed-threshold authorization, and extensible recovery under a single on-chain control plane.

The polished specification now makes two implementation-aligned points explicit. First, the externally meaningful identity is the deterministic AA address, while `accountId` remains an internal key used for storage, deterministic routing, and verifier compatibility. Second, the mixed-authorization path is account-centric: recovered EVM signers and Neo witnesses are normalized into one permission engine, and replay protection advances through an account-scoped nonce rather than signer-specific counters.

Taken together, these choices make Neo-AApractical as well as conceptually clean. One master contract can support many accounts without per-account deployment, users can discover and operate accounts through AA addresses or `.matrix` names, and recovery logic can evolve without redesigning the execution surface. Future work includes broader ecosystem support, privacy-preserving authorization, and more formal verification of the execution-context invariants that underpin the system's safety.